Linear Regression and Gradient Descent

Bhaya, AG*
Dept. of Computer Science and Technology,
University of North Bengal, Siliguri–10.

March 3, 2026

*✉ Electronic mail: ananya[dot]gb[at]nbu[dot]ac[dot]in

Contents

1 Gradient Descent **3**
1.1 Computational Complexity 3
1.2 Equation for Gradient Descent 5

2 Stochastic Gradient Descent **9**

1 Gradient Descent

Another way to minimize the cost (or loss) function is by using gradient descent.

Gradient Descent

Gradient descent is an *iterative algorithm*, which means we apply an update repeatedly until some criterion is met. We initialize the weights to something reasonable (*e.g.*, all zeros) and repeatedly adjust them in the direction of the steepest descent ($\max(-\nabla f)$).

1.1 Computational Complexity

Overall complexity of learning a linear regression model via *normal* equations is, $O(nm^2 + m^3)$; where X is a $m \times n$ matrix, and the

multiplication $X^T X$ has complexity $O(nm^2)$. To solve the system of equations of the form, $X^T X w = X^T y$, the complexity is $O(m^3)$.

As a result, using the normal equations (closed-form solution) to solve for linear regressional parameters, can be highly computationally expensive at times. In such cases one can optimize learning using **gradient descent**.

Gradient Descent	**Closed-form Solution**
Requires multiple iterations.	Non-iterative in nature.
Need to choose α.	No need for α.
Works well when n is large.	Slow if n is large (roughly $O(n^3)$).

Table 1: Comparison between Gradient Descent and Closed Solution.

Is gradient descent always better?

Normal equations cost roughly $O(nd^2+d^3)$, where as gradient descent costs $O(ndt)$ to run for t iterations. But gradient descent is not always the path ahead. In cases where n is small, gradient descent is computationally expensive.

1.2 Equation for Gradient Descent

A parameter, say w_i, for a loss function $L(w)$, can be updated using,

$$w^{t+1} = w^t - \alpha \nabla_{w^t} L(w) = w^t - \alpha \frac{\partial L(w)}{\partial w^t} \tag{1}$$

In equation (1), alpha is called the **learning rate**. It is a hyper-parameter that determines the size of the steps taken to reach the minimum of the cost function $L(w)$.

What could be a correct choice α?

The value of α (learning rate) can be chosen at random, as one sees fit. But it is important to choose a feasible value of α. If α is too small a value, it might be computationally expensive to reach the local minima in terms of time.
If too big a value is chosen, the algorithm might overshoot across the local minima, thereby getting stuck in a loop reverting to and fro across the local minima.

In equation (1), the value of $L(w)$, the loss function is,

$$L(w) = \sum_{i=1}^{n} (y_i - w^T x_i)^2, \text{ and} \tag{2}$$

$$\nabla_{w_j} L(w) = -\sum_{i=1}^{n} (y_i - w^T x_i) x_{ij} \tag{3}$$

Problem 1

Let (x_i, y_i) be a data point, where $x_i \in \mathbb{R}^d$ is a feature vector and $y_i \in \mathbb{R}$ is the corresponding output. There are a total of m data points. The cost function $J(w)$ is given by,

$$J(w) = \frac{1}{2} \sum_{i=1}^{m} (y_i - w^T x_i)^2$$

where, $w = [w_1, w_2, \ldots, w_d]^T \in \mathbb{R}^d$ is the vector of weights. What is teh correct expression for the derivative $\dfrac{\partial J}{\partial w_j}$?

Solution 1

The gradient is of the form, $\nabla J(w^t) = \left[\frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2}, \dots \frac{\partial J}{\partial w_d}\right]$.

$$
\begin{aligned}
\frac{\partial J}{\partial w_j} &= \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{i=1}^{m} (y_i - w^T x_i)^2 \\
&= \frac{1}{2} \sum_{i=1}^{m} 2(y_i - w^T x_i) \frac{\partial}{\partial w_j} (y_i - w_1 x_{i1} - \dots w_j x_{ij} - \dots \\
&= \sum_{i=1}^{m} (y_i - w^T x_i)(-x_{ij}) \\
\Rightarrow \frac{\partial J}{\partial w_j} &= -\sum_{i=1}^{m} (y_i - w^T x_i) x_{ij}
\end{aligned}
$$

2 Stochastic Gradient Descent

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is an iterative optimization algorithm used to minimize loss functions in machine learning by updating model parameters (weights) using only one, or a few, randomly selected data points at a time.

The loss function, $L(w)$, for SGD is defined by,

$$L(w) = \frac{1}{D_{\text{train}}} \sum_{(x,y) \in D_{\text{train}}} L(x, y, w) \tag{4}$$

Instead of looping through all the training examples, to compute a single gradient, and make one step, SGD loops through the examples (x, y) and updates the weights w based on each example. Each

update is not as precise, since it is based on one example, but we can make many more updates this way.

One could randomize the order in which they loop over the training data in each iteration. This is important, for instance, if your traning data has all the positive examples first then the negative examples afterwards.

GD and SGD trade off.

No doubt that SGD can run much faster than normal GD. But GD moves directly downhill following only one direction (opposite to $\max \nabla f$). SGD, on the other hand, its direction is determined by a few pre-selected points. As a result it takes step in a noisy direction, but moves downhill on average.